# CIS241

## System-Level Programming and Utilities

### C - Header Files

**Erik Fredericks, frederer@gvsu.edu**

**Fall 2025**

> **Based on material provided by Erin Carrier, Austin Ferguson, and Katherine Bowers**

# Time to make our code *reusable*

**Let's say we want to reuse a function, or our incredible struct from last time...**

```c
typedef struct Player {
  int roomID;
  int health;
  char* name;
} Player;
```

**How do we reuse this *across projects*?**

# Header files!

Quote indicate the file is local, angle brackets indicate it is installed on your system!

**Header files contain C code, filenames, end in .h**

**Where have we seen these before?**

- The files we've been including, example:
  - `#include <stdio.h>`

**For system headers, we use**

```
#include <file.h>
```

**For local headers that we write:**

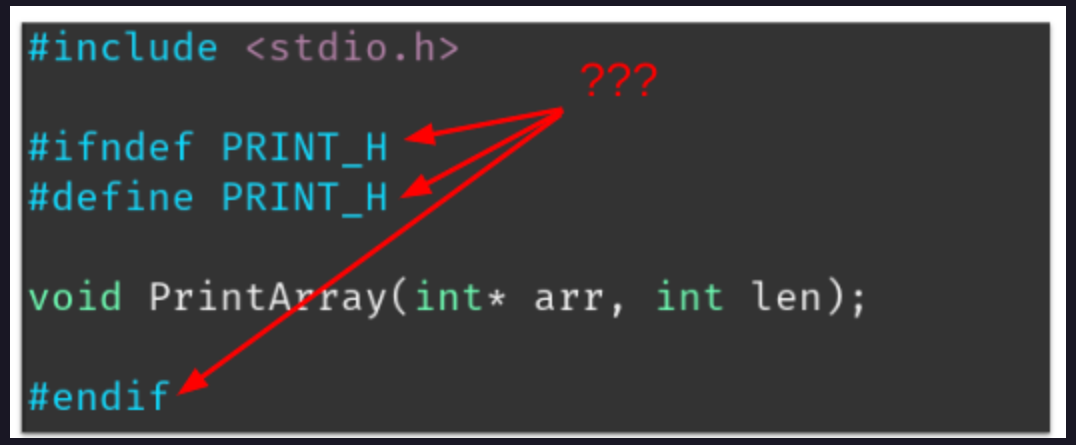
```
#include "file.h"
```

# Traditionally...

**Traditionally, header files contain structs, variables, and function prototypes**

- (You can put function definitions in headers, there are tradeoffs)

# Example header file




**This extra stuff is a header guard**

- `#ifndef` checks to see if macro (`PRINT_H` here) is defined, if so, it cuts out all code between it and `#endif`

**Because we define the macro inside this `if`, we ensure we have, at most, one copy of this code!**

- `#pragma once` is a modern alternative

# Function definition (i.e., the code?)

**The code goes in a *separate* `.c` file with the same name**

**`print.c`:**

```c
#include "print.h"

void PrintArray(int* arr, int len) {
    printf("[");
    for (int i = 0; i < len; i++) {
        if (i != 0) printf(" ");
        printf("%d", *(arr + i));
        if (i != len - 1) printf(",");
    }
    printf("]\n");
}
```

# Using the header file

```c
#include <stdio.h>
#include "print.h"

int main() {
    int arr_1[] = {1, 2, 3, 4, 5, 6, 7};
    PrintArray(arr_1, 7);
    return 0;
}
```

**However...**

# However!

**How do we tell `main.c` about `print.c`?**

- Compile it in!
- `gcc main.c print.c -o printer.o`

# Saving intermediate builds

**What if we don’t want to compile all files each time?**

- E.g., maybe print.c rarely changes, no reason to recompile.

**We can compile them separately using object files:**

- `gcc -c print.c`
  - (creates `print.o`)
- `gcc program.c print.o -o printer.o`

**This is called linking files**

- Ever have a linker error?

# Where are includes located?

**A local include like: `#include "file.h"` will first look in current directory**

- System also has some go-to directories that can be checked.

**We can also specify paths to check for files in our gcc call using `-I`**

**Example:**

- `gcc -Iother_proj/headers`
  - Note the lack of space after `I`

# Preprocessor directives (i.e., more-of)

**Remember: these operate on text, they do not consider code context**

**List (part 1):**

- `#include <header.h>` **-> paste contents of header.h**
- `#define NAME X` **-> Replace instances of NAME with X**
- `#define NAME(a) printf(“a”);` **-> Can also take arguments**
- `#undef NAME` **-> Undefines NAME**

# Preprocessor directives (i.e., more-of)

**Remember: these operate on text, they do not consider code context**

**List (part 2):**

- `#if X` **-> Includes following code if X is not 0**
- `#elsif` **and** `#else` **-> Go with if**
- `#endif` **-> Ends if / elsif / else blocks**
- `#ifdef X` **-> Like if; includes code if X is defined**
- `#ifndef X` **-> Includes if X is NOT defined**